

Experiment-8-Implementation of Hebb's Learning Rule

Aim

To implement Hebb's Learning Rule .

Theory

Hebbian learning is one of the earliest learning rules in artificial neural networks. It is based on the principle:

"Neurons that fire together, wire together."

According to Hebb's rule, if an input neuron and output neuron activate together, the connection weight between them increases.

The mathematical representation of Hebb's rule is:

$$W_{new} = W_{old} + X \times t$$

where

- X = Input vector
- t = Target output
- W = Weight vector

Algorithm

1. Start the program.

x_1	x_2	Output
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1

2. Initialize weight vector $w = 0$ and bias $b = 0$.
3. Provide input training vectors and their target outputs.
4. For each training pattern:

(a) Update weight using

$$w_i = w_i + x_i \times t$$

(b) Update bias using

$$b = b + t$$

5. Repeat the process for all training samples.
6. Display the final weight vector and bias.
7. Stop.

Program

```
1
2 import numpy as np
3
4 import numpy as np
5
6 # Training data (bipolar inputs)
7 # Example: AND logic pattern
8 inputs = np.array([
9     [1, 1],
10    [1, -1],
11    [-1, 1],
12    [-1, -1]
13 ])
14
15 # Target outputs (bipolar)
16 targets = np.array([1, -1, -1, -1])
17
18 # Initialize weights and bias
19 weights = np.zeros(2)
20 bias = 0
21
22 # Hebbian learning rule
23 for i in range(len(inputs)):
24     weights = weights + inputs[i] * targets[i]
25     bias = bias + targets[i]
26
27 print("Final_Weights:", weights)
28 print("Final_Bias:", bias)
```

Output

Final Weights: [2 2]
Final Bias: -2

Conclusion

Hebbian learning rule was successfully implemented .